

Lesson Transcript Controlling Model Complexity

What do we know from E-tivity 1?

In E-tivity 1 we learned that there is an elegantly simple relationship between the amount of training data we have available and the complexity of the model we can afford. We saw that, as we wish to use a more complex model, we need to increase the amount of training data to ensure we do not coincidentally stumble upon a model that performs great on training, or in-sample data, but disastrously on out of sample data. In other words, we will always need to balance two things: on the one hand we will want to use a suitably complex model to capture the important trends in our data. On the other hand, we will need to control the flexibility of our model to prevent capturing trends that arise because of noise or the particular probabilistic choice of our dataset. We call the latter overfitting.

A classification example

To illustrate the issue, let's look at an example. Let's assume we have a number of data points belonging to the blue and orange class. Now, if we want to be the best student in the class, we might want to create a rather wiggly line that puts all blue data points on one side, and all orange data points on the other side of the line. Using the mathematical representation of the line, which is effectively our machine learning model, we could determine if a new point falls above or below the line and thus predict its class membership.

Whilst this result looks great, we are aware that we should really limit model complexity in accordance with the amount of available training data so we decide to check how we did by collecting some new data. And lo and behold, we find that our wiggly model misclassifies many of the new data points.

So we decide to try again, but now with a simpler model. The unavoidable consequence is that we will make some errors on our training data, but the general result will be that errors on out-of-sample data are much reduced. In other words, the results obtained in training generalise better to the out-of-sample reality of using our model with new, yet unseen data.

A regression example

We have thus far looked at classification problems as the theory of generalisation was derived for classification. Just to show that the exact same holds for a regression, consider the following problem in which it is our task to predict the value of a new data point accurately.

Again our first reaction is to create a wiggly model that predicts each point perfectly. In this case, when given an X coordinate, the model, which can be seen as a complex mathematical function of X, will predict the corresponding Y value. When we momentarily assume that we know the real process that generated this data and plot this process in the same graph, we can see that our wiggly machine learning model makes rather large errors on some of the data points.

So again we use our knowledge that simpler is often better to create a smoother function. Even though this function is not able to predict the Y-value of most of the training data points without error, we can expect better out-of-sample performance, which is illustrated by again plotting the normally unknown function that generated these data points. And now we see that, although the model we chose may be a little simpler than the process itself, the error we can expect to make when predicting the Y-value of new data points, is much reduced.

These examples show that the art of machine learning consists of choosing a sufficiently expressive model on the one hand, and controlling the flexibility of the model on the other hand. And in this E-tivity we will look at both.

Non-linear Data transformations

Model complexity can be controlled in various ways, but many of the approaches that you will see when diving into more complex machine learning models are based on non linear data transformations. The core idea in performing these data transformations, is that data, that cannot be separated in the original feature space, often becomes separable, or at least better separable in a transformed feature space. To see this, consider the following example. The dataset in this plot contains examples of two classes, with the examples of the blue squares class all far away from the origin relative to the orange circle datapoints. When we consider a linear classifier, such as logistic regression, to correctly classify these data points into the two classes, we encounter an issue. After all, a linear classifier will result in a linear decision boundary, in this 2D case a line, and there is not a single line we can draw through this dataset to satisfactorily separate the two classes.

This is where non-linear data transformations come to the rescue. In this artificial example, we can see that the orange circle data points tend to be close to the origin and the blue squares tend to be further away. Hence, if we can create a decision boundary based on this distance from the origin, we could potentially end up with a pretty good classifier.

To get the distance of a datapoint to the origin, we take the sum of squares of the two coordinates and then take the square root. The square root achieves that the obtained distance has the same units as the original grid units. If we are just interested in comparing distances from the origin, as we are here, we can leave this step out. If we add a feature which is the sum of the two squared coordinates as a third dimension, we will see that in this new 3D space, data points that are far away from the origin, will end up lying at a higher level in our new space. This is great, because now we can find a decision boundary that perfectly separates the two classes. Note that for the 2D space our decision boundary was a line. As we now have a 3D data space, the decision boundary becomes a surface and we could choose a surface parallel with the ground plane at a height of 7 to separate the two classes perfectly.

The equation of this surface is then that $x_1^2 + x_2^2 = 7$ as implied by our choice for the third axis, and this happens to be the equation for a circle in our original 2D data space with a squared radius of 7.

Hence, we see that the addition of an extra feature as a non-linear transformation of the original features, can result in a non-linear decision boundary in our original data space, even when using a linear classifier such as logistic regression.

Whilst we have to resort to explicit data transformations when tackling such problems with logistic regression, this is not required in general. More complex machine learning models such as neural networks and support vector machines perform these types of non-linear transformations ‘under the hood’ so an explicit transformation is not required.

Transformations and model complexity

So now we know that data transformations, whether done explicitly on the input features of a model or implicitly inside the model, are potentially beneficial, let’s have a further look at the implications of such transformations for model complexity. For this we will consider this cloud of points with the aim of fitting a regression curve to these points. Without data transformation, we see that we do not

capture the exponential characteristic of the data points. We call this behaviour where models insufficiently capture the trends in our model “underfitting”. Having found such an underfitting model, we decide on using the square of the original feature in addition to the feature, x_1 . Although in general it can be difficult or even impossible to numerically determine the complexity of a model in terms of its VC dimension, for linear models this is straightforward as the VC dimension of these simple models is equal to the number of free parameters, or weights, which is equal to the number of features, plus one because the bias term acts as a feature with constant value 1. And this allows us to immediately see what the effect of the non-linear transformation is on complexity. In this example, we only added one feature and our VC dimension went from 2 to 3. However, suppose we are still not content with the error we make in training and we decide to add, say, all powers of the original feature between 3 and 5, we end up with a model with a VC dimension of 6. Hence, compared to our original model with a VC dimension of 2, our data requirement is now three times higher. This may seem like an acceptable increase in data requirements, but now assume you have 10 features, each of which you want to apply this same data transformation to, including all the cross terms. Suddenly your VC dimension would increase from 11 for a model using only the original features to 8009! (For those interested, see this link: <https://mathoverflow.net/questions/225953/number-of-polynomial-terms-for-certain-degree-and-certain-number-of-variables> for an explanation).

How to control model complexity?

All of this suggests, we may want to tune our models to get exactly the right level of complexity for the training dataset available, and this is where regularisation comes in. Regularisation can be thought of as the formalisation of a concept that has been known for quite some time as Occam’s Razor. Occam was a 13th century Franciscan friar and philosopher who is credited with formulating the principle that a simpler explanation is more likely to be correct than a more complex explanation. And this is exactly what regularisation tries to achieve: with a so-called regulariser we can penalise complex models, which in the case in hand we can think of as models using more non-linear transformations of the original feature. A suitable amount of regularisation will result in a non-zero error rate in training, but generally a much improved performance on out-of-sample data.

In this E-tivity we will focus on a specific type of regularisation called weight regularisation. This type of regularisation adds a penalty to the optimisation objective. Where so far we have tried to find the set of weights that minimise the in-sample error, we will now add a penalty term to this minimisation goal that penalises weights that are large. The severity of the penalty can be tuned with the parameter λ and finding the optimum λ is the core goal in weight regularisation. It may not be immediately obvious that this penalty results in simpler models. The intuition here is that, if we make our model more complex by adding lots of non-linear features, small weights will result in these non-linearities only be used sparingly. In some cases, and we will see this case in this E-tivity, weights will even be encouraged to be equal to zero, which means that we can very specifically choose the non-linear features that really matter in capturing the trends of interest, and thus only add relevant complexity to our model.

What does dVC tell us?

In terms of the theory of generalisation and the notion that model complexity negatively impacts on the gap between in-sample and out-of-sample performance, the so-called generalisation error, we can think of a regulariser as trying to find the optimal VC dimension such that the in-sample performance is reasonable, but without an overly high generalisation error such that the out-of-sample performance is unacceptable. Conceptually, we try to find the sweet spot in this graph which depicts the theoretical in-sample error in blue which will decrease as model complexity increases. As

can be seen, this error eventually reduces to zero as the model complexity is increased. This is expected as an infinitely flexible model can perform perfectly on every single training data sample. However, we also see the generalisation error in red increases as model complexity increases. As the out-of-sample error is the sum of these two error terms, the out-of-sample error will initially decrease as we increase model complexity, but then start increasing again as our model becomes too complex. Thinking back of the earlier examples, it is in the left hand part of this diagram where we will find models that are underfitting. The right hand part of this diagram is associated with models that are overfitting and our goldy-locks just-right model can be found in between these two extremes.

Validation

But how do we arrive at the optimal amount of regularisation? For this, we will use so-called validation data, some data never seen by the model during training to obtain an estimate of the probable out-of-sample performance. To get this data, we will split up our data set into a training data set and validation data set. With the training data set, we train a number of models with differing amounts of regularisation. We then use the validation data set, which the model has not seen during training, to get an estimate of out-of-sample performance and then choose the model that performs best.

This is a widely used approach, which has one major disadvantage. We previously saw that the VC theory demonstrated that we can get a model that likely generalises well by using as much training data as possible. But we also saw that the Hoeffding bound tells us that the validation dataset will be a better estimator for the likely out-of-sample performance as it grows. In other words, we would like both data sets to be as large as possible, but in practice you generally have a limited budget in terms of available data.

Cross validation

It turns out that there is a very elegant solution to this problem that may seem like a little bit of cheating. This method is called Cross validation and we will introduce it here with an example. Suppose you have a small dataset and you don't want to split the data into a training and validation set as you are afraid your models will otherwise need to be too simple to capture the trends of interest. Instead we split this dataset into three equal parts, called folds. Two folds will be used for training, the third for validation.

The first time you train your models on folds 1 and 2, and you validate your models on fold 3. You now store all the results and repeat this process, but now you train your model on folds 1 and 3, and you validate your models on fold 2. Again you store the results and repeat the whole process once more, but now you use folds 2 and 3 for training and fold 1 for validation.

You will now have a family of models and aggregated over all models, you have used all three folds for training as well as having used all three folds for validation. As you have kept track of all validation errors, you can calculate average validation errors across all three folds for each of your models, and then pick the model with the lowest average validation error.

K-fold Cross Validation

This trick is called K-fold cross validation, where K stands for the number of folds you choose. Typical choices are 5 or 10 folds. In summary, with a dataset of N examples and the use of K folds, you train on all folds but one. You then used this last fold to obtain an estimate of the out-of-sample error. By repeating this K times, each fold will be used once to estimate the validation error and all folds will

be used to train K-1 models. By averaging the performance for a specific value of λ across all folds, you can now approximate the out-of-sample error much better as you effectively have used all available data. Moreover, the chosen λ is likely better as it has been chosen making use of all available training data.

Finally, this approach is not limited to use when you wish to choose the optimal amount of regularisation. This approach can be used to tune any other hyperparameter in your model.

Summary

And with that we have discussed two aspects of machine learning that are crucially important for any practical use of machine learning algorithms. We have discussed the importance of non-linear data transformations to allow machine learning models to capture the often non-linear trends of interest in the data. We have seen how these non-linear transformations can be applied explicitly in logistic regression. More complex machine learning models will perform these data transformations internally. We saw that, whilst such non-linear transformations allow us to fit more complex trends better, they also give rise to higher VC dimensions which, for the same amount of training data, will result in a higher generalisation error. As the expected out-of-sample error is equal to the in-sample error plus the generalisation error, we want to keep the latter low. And this can be achieved with regularisation which is a means of tuning the complexity of a machine learning model to the available data. Whilst the VC theory is really useful to show that regularisation is a good idea, it does not show us how much regularisation to apply. For this we need to create an estimate of our out-of-sample performance and for this we create a validation data set. This validation data set is never used during training and can therefore be seen as an unbiased estimate of what out-of-sample data may look like. Hence, we can use it to estimate our out-of-sample performance. The Hoeffding bound tells us that the larger the validation dataset, the better we will be able to estimate the out-of-sample performance. As data is generally a scarce commodity this results in a dilemma. A larger validation data set per definition results in a smaller training data set. But this is where K-fold cross validation came to the rescue as a smart strategy to use all data for training as well as using all data for validation. Of course any advantage comes with its disadvantages and in this case that is that you have to repeat model training K times.